



in collaboration with



STW7086CEM: Data Management Systems

Submitted To

Name: Ritesh Singh

Submitted by

Name: Aishwarya Rai
Student Id: **250061**

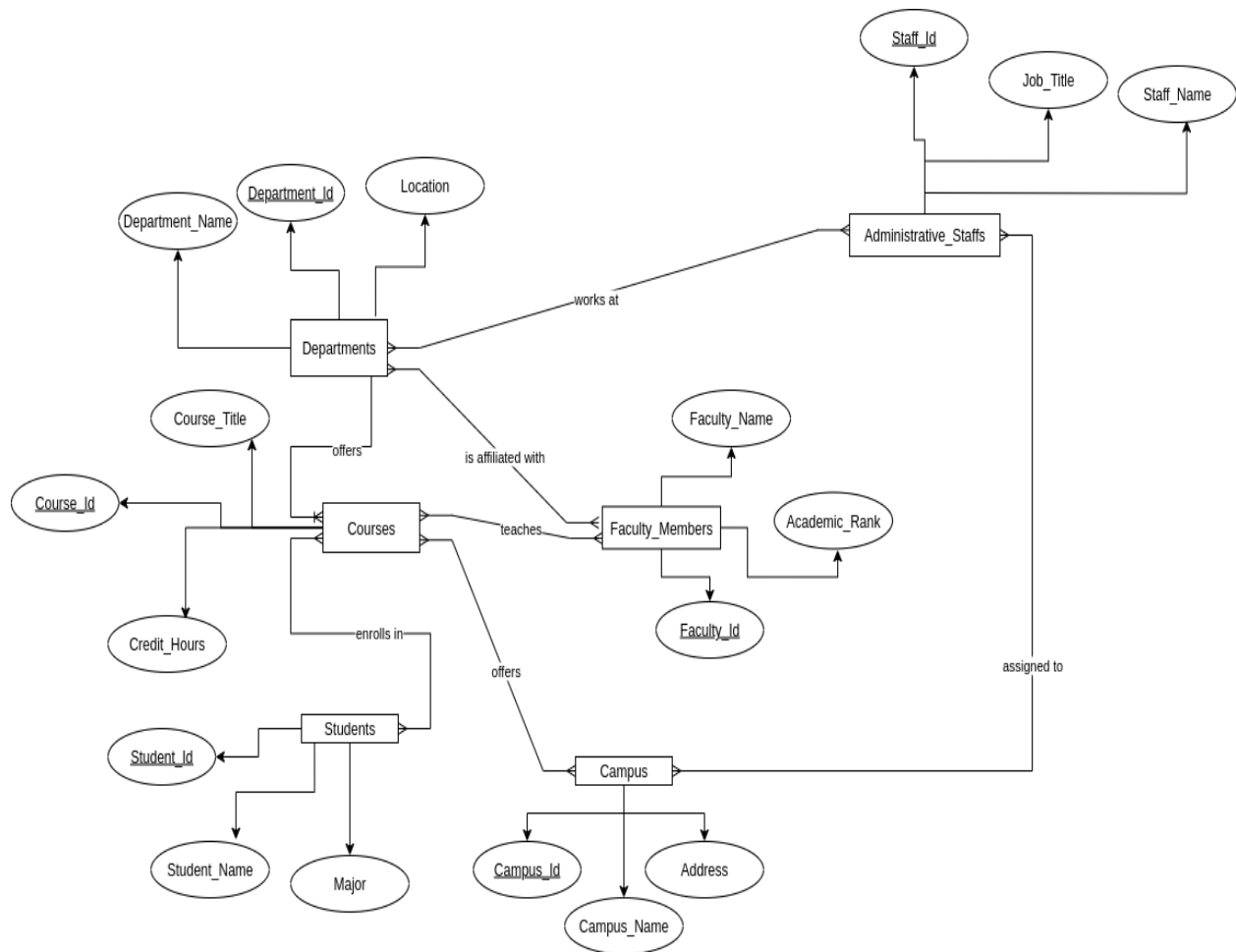
CoventryUniversity Student Id: **16542402**

Name: Sandesh Tamang
Student Id: **250060**

CoventryUniversity Student Id: **16542804**

Part A – Entity Relationship Modeling

i) ER Diagram Description



The above Entity-Relationship (ER) diagram is of the university database, indicating entities, their attributes, identifiers, and the cardinality. The cardinality of relationships and their nature are described below:

1. DEPARTMENT --- OFFERS --- COURSE

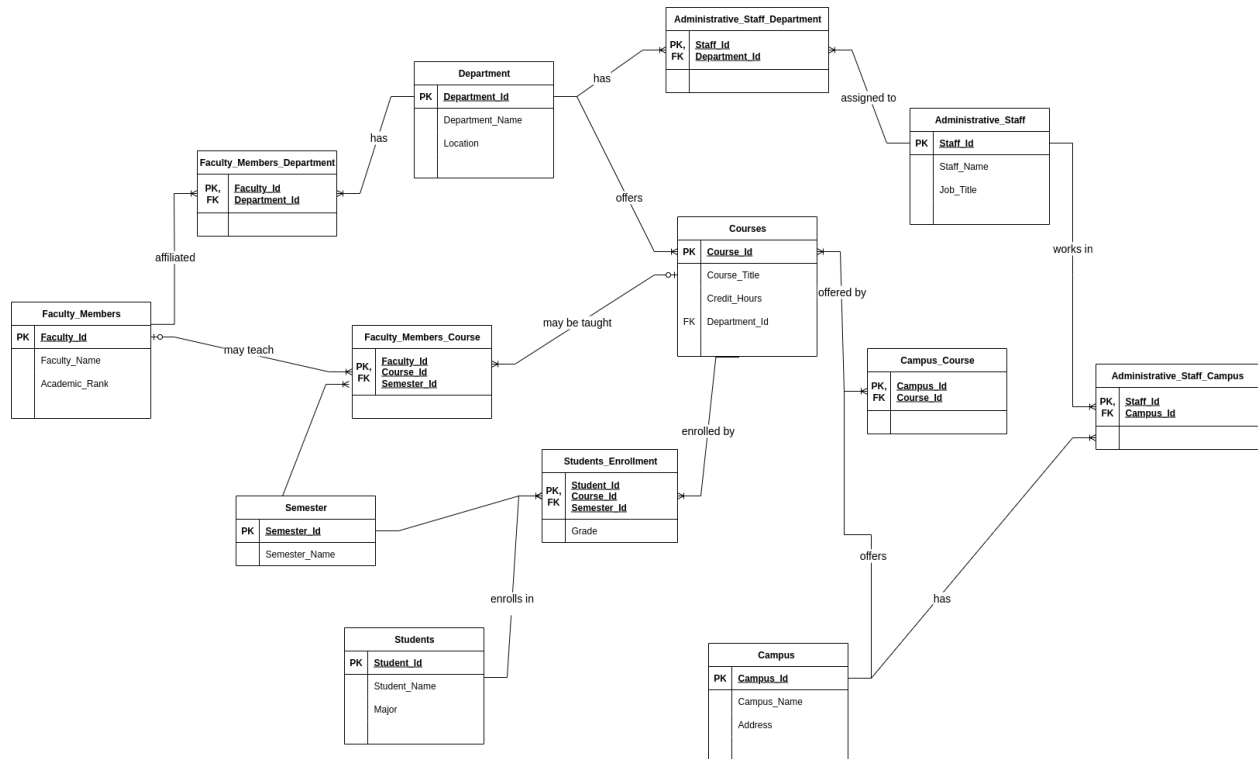
- **Cardinality:** One-to-Many (1:N)
- **Nature:**
 - A **Department** **must** offer at least one **Course** (mandatory, (1, N)).
 - A **Course** **must** be offered by exactly one **Department** (mandatory, (1, 1))

2. Faculty member --<is affiliated with>-- Department

- **Cardinality:** Many-to-Many (M:N) - Requires an associative entity.
- **Nature:**
 - A **Faculty member** **must** be affiliated with at least one **Department** (mandatory, (1, N)).

- A **Department** **must** have at least one **Faculty member** affiliated (mandatory, (1, N)).
3. **Faculty --<teaches>-- Course**
- **Cardinality:** Many-to-Many (M:N) - Requires an associative entity.
 - **Nature:**
 - A **Faculty** member **may** teach courses (optional, (0, N)).
 - A **Course** **may** be taught by faculty members (optional, (0, N)).
4. **Student --<enrolls in>-- Course**
- **Cardinality:** Many-to-Many (M:N) - Requires an associative entity.
 - **Nature:**
 - A **Student** **must** enroll in at least one **Course** (mandatory, (1, N)).
 - A **Course** **must** have at least one **Student** enrolled (mandatory, (1, N)).
5. **Campus --<offers>-- Course**
- **Cardinality:** Many-to-Many (M:N) - Requires an associative entity.
 - **Nature:**
 - A **Campus** **must** offer at least one **Course** (mandatory, (1, N)).
 - A **Course** **must** be offered at least one **Campus** (mandatory, (1, N)).
6. **Administrative_Staff --<assigned to>-- Department**
- **Cardinality:** Many-to-Many (M:N) - Requires an associative entity.
 - **Nature:**
 - An **Administrative_Staff** member **must** be assigned to at least one **Department** (mandatory, (1, N)).
 - A **Department** **must** have at least one **Administrative_Staff** member assigned (mandatory, (1, N)).
7. **Administrative_Staff --<assigned to>-- Campus**
- **Cardinality:** Many-to-Many (M:N) - Requires an associative entity.
 - **Nature:**
 - An **Administrative_Staff** member **must** be assigned to at least one **Campus** (mandatory, (1, N)).
 - A **Campus** **must** have at least one **Administrative_Staff** member assigned (mandatory, (1, N)).

ii) Relational Tables from ER Diagram



Above figure shows the relational tables generated from the ER diagram, using schema notation.

Transformation Rules Used:

- Rule 1 (Strong Entities):** Each strong entity (Department, Course, Faculty_Members, Student, Campus, Administrative_Staff) is transformed into a relational table. Its attributes become columns, and its identifier becomes the primary key.
- Rule 2 (1:N Relationships):** For the 1:N relationship between **Department** and **Course** (where a Course belongs to one Department), the primary key of the "one" side (**department_ID** from **Department**) is added as a foreign key to the "many" side (**Course** table).
- Rule 3 (M:N Relationships):** Each Many-to-Many relationship (Faculty_Members_Department, Faculty_Members_Course, Students_Enrollment, Campus_Course, Administrative_Staff_Department, Administrative_Staff_Campus) is transformed into a new associative (or junction) table. The primary key of these new tables is the composite key formed by the primary keys of the entities involved in the relationship.

Part B: SQL Programming

Task 1: Create Respective Tables and Data Insertion

```
-- =====
-- Create Category Table
-- =====

CREATE TABLE Category (
    categoryCode VARCHAR2(10) PRIMARY KEY,    -- Primary key constraint
    title VARCHAR2(50) NOT NULL               -- NOT NULL constraint
);

-- =====
-- Create User Table
-- =====

CREATE TABLE UserInfo (
    userId VARCHAR2(20) PRIMARY KEY,          -- Primary key
    name VARCHAR2(100) NOT NULL,              -- User's full name
    emailAddress VARCHAR2(150) NOT NULL UNIQUE -- Email must be unique
);

-- =====
-- Create Music Table (References Category)
-- =====

CREATE TABLE Music (
    musicId VARCHAR2(10) PRIMARY KEY,          -- Primary key
    title VARCHAR2(200) NOT NULL,              -- Song title
    categoryCode VARCHAR2(10) NOT NULL,        -- Foreign key to Category
    costPerDownload NUMBER(5,2) NOT NULL,      -- Price with 2 decimal places

    -- Foreign key constraint
    CONSTRAINT fk_music_category
        FOREIGN KEY (categoryCode)
        REFERENCES Category(categoryCode)
);
```

```

-- =====
-- Create DownloadMusic Table (References User and Music)
-- =====

CREATE TABLE DownloadMusic (
    userId VARCHAR2(20) NOT NULL,           -- Foreign key to User
    musicId VARCHAR2(10) NOT NULL,          -- Foreign key to Music
    downloadDate DATE NOT NULL,             -- When download occurred

    -- Composite primary key (all three columns together)
    PRIMARY KEY (userId, musicId, downloadDate),

    -- Foreign key constraints
    CONSTRAINT fk_download_user
        FOREIGN KEY (userId)
        REFERENCES UserInfo(userId),

    CONSTRAINT fk_download_music
        FOREIGN KEY (musicId)
        REFERENCES Music(musicId)
);

```

```

SQL> CREATE TABLE Category (
    categoryCode VARCHAR2(10) PRIMARY KEY,    -- Primary key constraint
    title VARCHAR2(50) NOT NULL               -- NOT NULL constraint
)

```

Table CATEGORY created.

Elapsed: 00:00:00.016

```

SQL> CREATE TABLE UserInfo (
    userId VARCHAR2(20) PRIMARY KEY,          -- Primary key
    name VARCHAR2(100) NOT NULL,              -- User's full name
    emailAddress VARCHAR2(150) NOT NULL UNIQUE -- Email must be unique...
)

```

Show more...

Table USERINFO created.

Elapsed: 00:00:00.017

```

SQL> CREATE TABLE Music (
    musicId VARCHAR2(10) PRIMARY KEY,          -- Primary key
    title VARCHAR2(200) NOT NULL,              -- Song title
    categoryCode VARCHAR2(10) NOT NULL,        -- Foreign key to Category...
)

```

Show more...

Table MUSIC created.

Elapsed: 00:00:00.019

```

SQL> CREATE TABLE DownloadMusic (
    userId VARCHAR2(20) NOT NULL,              -- Foreign key to User
    musicId VARCHAR2(10) NOT NULL,             -- Foreign key to Music
    downloadDate DATE NOT NULL,                -- When download occurred...
)

```

Show more...

Table DOWNLOADMUSIC created.

Elapsed: 00:00:00.018

```

62 -- Data Insertion
63 -- =====
64
65 -- Categories - Multiple values in single INSERT
66 INSERT INTO Category (categoryCode, title)
67 VALUES
68     ('M011', 'Love'),
69     ('M012', 'Pop'),
70     ('M013', 'Rock');
71
72 -- Users - Multiple values in single INSERT
73 INSERT INTO UserInfo (userId, name, emailAddress)
74 VALUES
75     ('wayne10', 'Wayne, R', 'wayne@hotmail.co.uk'),
76     ('santos17', 'Santos, C', 'santos@ntl.co.uk'),
77     ('hary05', 'Hary, M', 'hary@freemove.co.uk'),
78     ('margot9', 'Margot, C', 'margot9@msn.co.uk'),
79     ('mount77', 'Mount, M', 'mount@hotmail.co.uk'),
80     ('nancy91', 'Nancy, L', 'nancy91@tesco.co.uk');
81
82 -- Music - Multiple values in single INSERT
83 INSERT INTO Music (musicId, title, categoryCode, costPerDownload)
84 VALUES
85     ('M001', 'Born to run', 'M013', 0.99),
86     ('M002', 'Crawling', 'M013', 1.99),
87     ('M003', 'You're Beautiful', 'M011', 1.49),
88     ('M004', 'Summer of 69', 'M011', 1.79),
89     ('M005', 'Crazy Train', 'M013', 1.50),
90     ('M006', 'Yellow Submarine', 'M012', 1.10),
91     ('M007', 'Got to be there', 'M012', 0.89);
92
93 -- Download records - Multiple values in single INSERT
94 INSERT INTO DownloadMusic (userId, musicId, downloadDate)
95 VALUES
96     ('wayne10', 'M002', DATE '2021-05-03'),
97     ('margot9', 'M005', DATE '2022-05-01'),
98     ('santos17', 'M002', DATE '2021-05-06'),
99     ('margot9', 'M001', DATE '2022-05-06'),
100     ('wayne10', 'M003', DATE '2022-08-01'),
101     ('mount77', 'M004', DATE '2022-08-02'),
102     ('nancy91', 'M007', DATE '2021-09-05');
103
104 -- Commit all changes
105 COMMIT;
106

```

```
SQL> INSERT INTO Category (categoryCode, title)
VALUES
    ('M011', 'Love'),
    ('M012', 'Pop'),...
Show more...
```

3 rows inserted.

Elapsed: 00:00:00.022

```
SQL> INSERT INTO UserInfo (userId, name, emailAddress)
VALUES
    ('wayne10', 'Wayne, R', 'wayne@hotmail.co.uk'),
    ('santos17', 'Santos, C', 'santos@ntl.co.uk'),...
Show more...
```

6 rows inserted.

Elapsed: 00:00:00.023

```
SQL> INSERT INTO Music (musicId, title, categoryCode, costPerDownload)
VALUES
    ('M001', 'Born to run', 'M013', 0.99),
    ('M002', 'Crawling', 'M013', 1.99),...
Show more...
```

7 rows inserted.

```
SQL> INSERT INTO DownloadMusic (userId, musicId, downloadDate)
VALUES
    ('wayne10', 'M002', DATE '2021-05-03'),
    ('margot9', 'M005', DATE '2022-05-01'),...
Show more...
```

7 rows inserted.

Elapsed: 00:00:00.016

```
SQL> COMMIT
```

Commit complete.

Elapsed: 00:00:00.002

Task 2: Running Respective Queries

```
-- =====  
-- Task2: Queries  
-- =====
```

```
-- Query 1: All music ordered by title  
SELECT musicId, title, categoryCode  
FROM Music  
ORDER BY title;
```

	MUSICID	TITLE	CATEGORYCODE
1	M001	Born to run	M013
2	M002	Crawling	M013
3	M005	Crazy Train	M013
4	M007	Got to be there	M012
5	M004	Summer of 69	M011
6	M006	Yellow Submarine	M012
7	M003	You're Beautiful	M011

```
124  
125 -- Query 2: User download count for Pop or Rock music category  
126 SELECT COUNT(DISTINCT dm.userId) as "User_Download_Count", c.title  
127 FROM DownloadMusic dm  
128 JOIN Music m ON dm.musicId = m.musicId  
129 JOIN Category c ON m.categoryCode = c.categoryCode  
130 WHERE c.title IN ('Pop', 'Rock')  
131 group by c.title  
132 order by c.title;  
133
```

	USER_DOWNLOAD_COUNT	TITLE
1	1	Pop
2	3	Rock

```

133 |
134 -- Query 3: Music Downloads Count per category
135 SELECT COUNT(dm.musicId) as "Music_Download_Count", c.title as "Category_Title"
136 FROM DownloadMusic dm
137 JOIN Music m ON dm.musicId = m.musicId
138 JOIN Category c ON m.categoryCode = c.categoryCode
139 group by c.title
140 ORDER BY COUNT(dm.musicId) DESC;
141

```

	MUSIC_DOWNLOAD_COUNT	CATEGORY_TITLE
1	4	Rock
2	2	Love
3	1	Pop

```

142
143 -- Query 4: Categories with music download count > 1
144
145 SELECT c.title as "Popular_Category_Title"
146 FROM DownloadMusic dm
147 JOIN Music m ON dm.musicId = m.musicId
148 JOIN Category c ON m.categoryCode = c.categoryCode
149 group by c.title
150 HAVING COUNT(dm.musicId) > 1
151 ORDER BY COUNT(dm.musicId) DESC;
152

```

	POPULAR_CATEGORY_TITLE
1	Rock
2	Love

Part C: Sequential and distributed processing

Task 1. Analyzing Sales Data with SQL (Relational Database) to determine for each product, the number of products which were sold in each month of each year.

```
186 |  
187 | SELECT  
188 |     ProductNo,  
189 |     YearId,  
190 |     MonthId,  
191 |     SUM(Quantity) AS TotalProductsSold  
192 | FROM  
193 |     SalesRecords -- Using 'SalesRecords' as the table name  
194 | GROUP BY  
195 |     ProductNo,  
196 |     YearId,  
197 |     MonthId  
198 | ORDER BY  
199 |     ProductNo,  
200 |     YearId,  
201 |     MonthId;  
202 |
```

Justification for the SQL Statement:

- **SELECT ProductNo, YearId, MonthId, SUM(Quantity) AS TotalProductsSold:** We select the columns that define our groups (ProductNo, YearId, MonthId) and use the SUM() function to add up the Quantity for all items within each group. AS TotalProductsSold gives our calculated sum a clear name.
- **FROM SalesRecords:** This specifies that we are querying data from our SalesRecords table.
- **GROUP BY ProductNo, YearId, MonthId:** This is the core of the aggregation. It tells the database to collect all rows that have the same values for ProductNo, YearId, and MonthId into distinct groups. The SUM() function then operates on the Quantity values within each of these groups.
- **ORDER BY ProductNo, YearId, MonthId:** This sorts the final results, making it easy to see the totals for each product, year, and month in a structured manner

Task 2: MapReduce for Sales Data Analysis to determine for each product, the number of products which were sold in each month of each year.

When sales data exceeds single-server capacity, MapReduce provides distributed processing capabilities:

- Horizontal Scaling: Processes massive datasets across cluster nodes
- Automatic Recovery: Reassigns failed tasks to healthy nodes
- Concurrent Processing: Parallel execution reduces processing time
- Developer-Friendly: Abstracts distributed system complexities

Implementation Overview

Objective: Calculate monthly product sales totals from large-scale transaction data.

Input Format:

OrderNo,ProductNo,Price,Quantity,Sales,QtrId,MonthId,YearId

10107,2,85.7,30,2587,1,2,2003

10107,5,95.8,39,3879.49,1,2,2003

MapReduce Workflow

Stage 1: Mapping

- Input: Split dataset chunks across mapper nodes
- Process: Extract **ProductNo**, **YearId**, **MonthId**, and **Quantity** from each record
- Output: Key-value pairs where:
 - Key: (**ProductNo**, **YearId**, **MonthId**) → e.g., (2, 2003, 2)
 - Value: Quantity → e.g., 30

Mapper Output Example:

((2, 2003, 2), 30)

((5, 2003, 2), 39)

Stage 2: Shuffling (Automatic)

- Framework groups identical keys together
- Routes grouped data to appropriate reducers
- Ensures all quantities for same product/month/year reach one reducer

Reducer Input Example:

((5, 2003, 2), [39, 34])

Stage 3: Reducing

- Process: Sum all quantity values for each key
- Output: Final aggregated totals

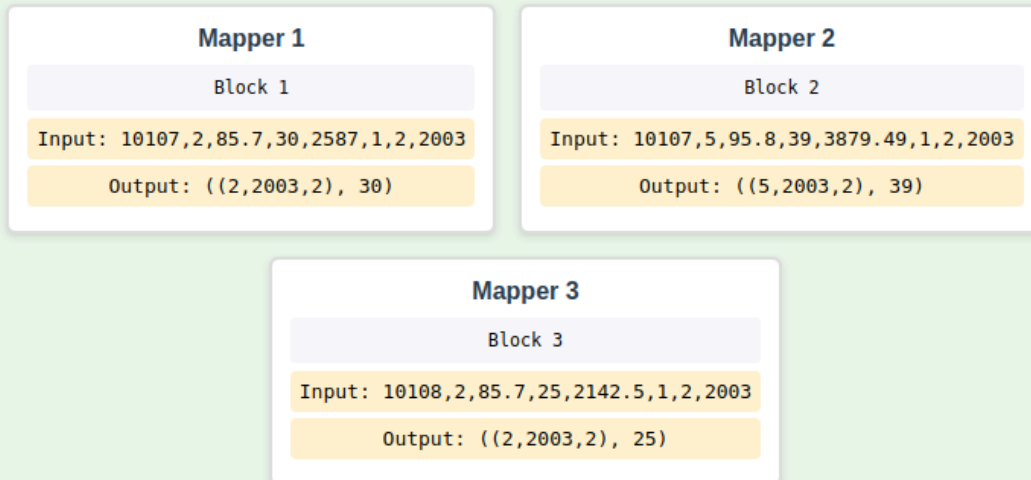
Reducer Output Example:

((5, 2003, 2), 73) // Total units sold for Product 5 in Feb 2003

Final Output

Results stored in a distributed file system showing total monthly sales per product, enabling efficient analysis of large-scale sales patterns. Below diagram clearly shows the working mechanism of MapReduce.

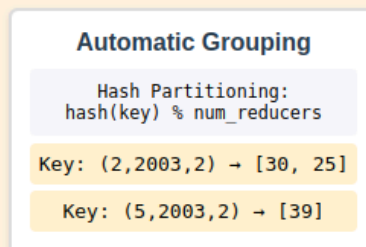
Map Phase: Parallel Processing



Each mapper extracts ProductNo, YearId, MonthId, and Quantity to create key-value pairs



Shuffle & Sort Phase: Grouping



Framework groups all values with identical keys and routes them to appropriate reducers

1 2 3 4 Reduce Phase: Aggregation

Reducer 1

Input: ((2,2003,2), [30, 25])

Sum: $30 + 25 = 55$

Output: ((2,2003,2), 55)

Reducer 2

Input: ((5,2003,2), [39])

Sum: $39 = 39$

Output: ((5,2003,2), 39)

Each reducer sums all quantity values for each unique product/month/year combination



Final Output: Results in HDFS

Aggregated Sales Data

```
part-r-00000:
(2,2003,2) 55
(5,2003,2) 39
```

Interpretation:
Product 2: 55 units in Feb 2003
Product 5: 39 units in Feb 2003

Final results stored across multiple part-files in HDFS, ready for business analysis

Part D: Research Report

Task 1: Research Report: ShopNow Database Migration Report: MySQL to MongoDB

Key Problems faced by ShopNow while using MySQL:

1. Rigid Schema Limitations

- **Fixed table structure** couldn't accommodate diverse product attributes
- **Schema changes** required downtime and complex migrations
- **Slow adaptation** to new product types and market demands

2. Poor Scalability

- **Vertical scaling** hit physical and cost limits
- **Horizontal scaling** extremely difficult due to complex joins
- **Data distribution** across servers became a bottleneck
- **Performance degraded** with growing user base and transactions

3. Slow Analytics

- **Complex queries** with multiple joins were computationally expensive
- **Real-time recommendations** suffered from processing delays
- **Marketing campaigns** couldn't be as immediate or responsive

4. Unstructured Data Challenges

- **User-generated content** (reviews, ratings) didn't fit tabular structure
- **Flexible data storage** required complex workarounds
- **Suboptimal performance** for dynamic user preferences and content

ShopNow's Strategic Migration to MongoDB(2019):

ShopNow's database transition was motivated by MongoDB's capability to resolve fundamental operational challenges:

1. **Dynamic Schema Architecture:** MongoDB employs a document-based structure using BSON format, eliminating rigid schema constraints. This **schema-agnostic** approach enabled ShopNow to incorporate new product attributes and categories without system downtime or complex database alterations. Product variations could be managed by simply adding fields to existing documents, accelerating development cycles and minimizing technical debt.
2. **Native Distributed Scaling:** MongoDB's built-in **sharding mechanism** facilitates automatic data partitioning across multiple server nodes. This distributed architecture allows ShopNow to expand computational capacity by incorporating additional hardware resources, maintaining

optimal performance levels while accommodating growing user bases and transaction volumes without architectural redesign.

3. **Unified Data Management:** The document-oriented model excels at managing heterogeneous data structures, including both organized and unstructured information. MongoDB enables **data embedding** within single documents (such as incorporating customer reviews directly into product records), eliminating expensive join operations and enhancing query performance.
4. **Optimized Query Performance:** MongoDB's denormalized storage approach typically delivers superior read/write performance for standard usage patterns. Its comprehensive query capabilities and aggregation pipeline support sophisticated analytical operations, enabling real-time data processing for personalized user experiences and targeted marketing strategies that were essential for ShopNow's competitive positioning.

Summary: ShopNow's migration from MySQL to MongoDB represented a calculated response to relational database constraints in high-growth environments. MongoDB's adaptive schema design, inherent scalability features, and versatile data handling capabilities established the technological infrastructure required for ShopNow's continued market expansion and superior customer service delivery.

Task 2: NoSQL queries

```
|  
// MongoDB NoSQL Queries for ShopNow Schema  
  
// 1. Retrieve all product details (title, category, price) where the price is less than $50  
db.products.find(  
  { price: { $lt: 50 } },  
  { title: 1, category: 1, price: 1, _id: 0 }  
);  
  
// 2. Update the product price of the product with productId "P003" to $75  
db.products.updateOne(  
  { productId: "P003" },  
  { $set: { price: 75 } }  
);  
  
// 3. Insert a new product with the following details: productId "P005", title "Smartwatch",  
category "Electronics", and price $149.99  
db.products.insertOne(  
  productId: "P005",  
  title: "Smartwatch",  
  category: "Electronics",  
  price: 149.99  
});  
  
// 4. Delete all products from the "Fashion" category  
db.products.deleteMany(  
  { category: "Fashion" }  
);
```